NAPCO Nucleus — User Manual

Requirement Management — On-demand pull-session workflow

1. How it works

Four input channels — Microsoft Teams chats, Microsoft Teams audio calls, email, and Google Drive — feed into one consolidated Word document called the *pull session*. You pull from each channel by command, in any order, in any combination. When the session has the material you want, one final command identifies the requirements, writes a verification Word doc, and drafts a single email to the client. The draft lands in your Outlook Drafts folder for manual review and send.

The system never sends mail on its own. Every email leaves your mailbox manually, with your explicit click of Send.

2. Start a pull session (optional)

The session doc is created automatically the first time you pull. If you want a fresh session — for example, you finished yesterday's batch and want today's pulls in their own doc — reset explicitly:

```
python -c "from tools._session_doc import reset; print(reset(label='today'))"
```

The previous session doc is archived under data/requirements/sessions/archive/ with its label and timestamp. The fresh one starts at data/requirements/sessions/current.docx.

3. Pull from each channel

3.1 Email

Pulls IMAP messages by sender, subject, and/or time window. Attached PDFs, .docx, and .txt files are extracted and appended to the email body so the LLM reads them inline.

```
python -m mail.pull_email [--from-sender ADDR]
[--subject TEXT]
[--last-minutes N]
[--from-time HH:MM --to-time HH:MM]
[--date YYYY-MM-DD]
```

Goal	Command
All emails in last 15 minutes	--last-minutes 15
From a specific sender, last hour	--from-sender "client@example.com" --last-minutes 60
Specific subject, last 30 min	--subject "budget" --last-minutes 30
Manual time window today	--from-time "3 PM" --to-time "5 PM"
All filters combined	--from-sender "alice@ex.com" --subject "Q3" --last-minutes 60

3.2 Teams chat

Reads messages from the local Teams desktop cache. Pick the chat by name (with CHAT_ALIASES), number, or full conversation_id. Optionally narrow to one sender's messages within that chat.

```
python -m teams.pull_chat (--name NAME | --number N | --id ID)
[--sender NAME]
[--last-minutes N]
[--from-time HH:MM --to-time HH:MM]
[--date YYYY-MM-DD]
```

Goal	Command
Last 15 min from a group	--name "ContiHosting" --last-minutes 15
Just one person's messages	--name "ContiHosting" --sender "Salman" --last-minutes 30
Manual window	--name "ContiHosting" --from-time "3 PM" --to-time "5 PM"
By chat number (when no alias)	--number 45 --last-minutes 30

3.3 Google Drive

Pulls and extracts content from files in your configured Drive folder. Handles PDF, plain text, .docx (modern Word), .doc (legacy Word, best-effort), and audio/video (transcribed via Groq Whisper).

```
python -m drive.pull_drive [--filename TEXT]
[--last-files N]
[--last-minutes N]
[--from-time HH:MM --to-time HH:MM]
[--date YYYY-MM-DD]
```

Goal	Command
Most recent file	--last-files 1
Most recent 3 files	--last-files 3
Files added in last 10 min	--last-minutes 10
By filename substring	--filename "budget"
Manual window	--from-time "3 PM" --to-time "9 PM"

3.4 Teams audio call

Records both the system loopback (the other party's voice) and your microphone as separate WAV tracks, so the transcript preserves speaker attribution. Three steps:

1. **Start recording.** `python -m teams.record_call` (leave it running in a terminal)
2. **Have the call.** The recorder captures everything until you stop it.
3. **Stop cleanly.** Either Ctrl+C in the recorder terminal, or run `python -m teams.stop_recording` from a different terminal.
4. **Transcribe and append.** `python pull_meeting.py`. Bangla and English are auto-detected; output is in English.

4. Inspect or manage the session

Open the live session doc directly:

```
data/requirements/sessions/current.docx
```

List the section titles already in it:

```
python -c "from tools._session_doc import status; print(status())"
```

5. Identify requirements and draft the email

One command does everything from this point:

```
python agent.py --task verify_session
```

Reads the session doc, identifies distinct client requirements, writes *Requirements Verification YYYY-MM-DD.docx*, drafts one email to the client (verification doc attached), and pushes the draft into your IMAP Drafts folder.

6. Review and send

1. Open Outlook (or any IMAP-aware mail client).
2. Go to your Drafts folder.
3. Find the draft (subject "Requirements Verification - YYYY-MM-DD").
4. Read the attached Word doc, review the email body, edit anything you'd like to phrase differently.
5. Click Send.

7. Troubleshooting

Symptom	Likely cause	Fix
"No chat matching X"	Empty title in IndexedDB	Add `X=NUMBER` to CHAT_ALIASES in .env
0 chat messages but you sent one	Teams cache hasn't flushed yet	Wait ~5 sec, retry
IMAP login failed	Gmail App Password wrong/expired	Regenerate in Google Account → Security
"GDRIVE_AUDIO_FOLDER_ID not set"	.env missing the variable	Add it to .env and ensure the service account has access to the folder
ModuleNotFoundError: faster_whisper	Dependencies not installed	`pip install -r requirements.txt`
Draft missing from Outlook Drafts	IMAP push failed (look at imap_error in tool output)	Open the .eml at `data/requirements/drafts/<date>/` directly
"No *_mic.wav recordings"	Recorder didn't capture or didn't stop cleanly	Re-run `python -m teams.record_call`; verify it prints Recording -> ...